Types from Fortran to Python via Opaque Pointers

Rohit Goswami · 2023-11-12 · python · fortran · numpy · f2py · ramblings

> *An exploration of the opaque object and pointer interface approach to unleash more modern features than those offered by the ISO Fortran standard.*

Background

Since this has been covered a few times before, just some quick pointers[^fn:1]. The ultimate context is to be able take nice, modern Fortran code with derived types and generate equally nice, user friendly, efficient Python wrappers via `f2py`.

Why?

Where the last post (and the test implementation in `f2py`) left off, the type shadowing approach for having exactly interoperable `bind(c)` derived types (and functions) was covered. However, there are several aspects of derived types which cannot be covered ("deep" types in the nomenclature of Pletzer et al. (2008)). A brief list:

- Types with allocatable members
- Types with other types as members
- Types which have bound proceedures
- Pointer elements
- Child types, in an inheritance hierarchy

Of these, the first three are probably the most common use cases, which need to be addressed by any serious attempt to bride Fortran and Python.

Scope

This post is just going to cover the development of Python examples facilitated by the pointer implementation where:

- A simple derived type, `point`, is manipulated via pointers

We will first demonstrate a Fortran example, followed by its call from `C`, before demonstrating its bridging `Python-C` code and finally a user-facing `python` example. The full code may be found in the examples folder and pull requests here. Typically the snippets here will have elided error handling to drive home the approach over the boilerplate / best practices [^fn:2].

Approaches to an automated method

The main idea here is a separation of concerns at three levels:

- Fortran code handles its own memory / functionality
  - Wrappers are exposed at two levels:
    - One layer to provide pointer "handles" to each part of the type.
    - Another outer layer connecting to `bind(c)` proceedures.

- `C` bindings call the `bind(c)` proceedures and pass handles (opaque pointers) as `void*`.
- At the `python` level, these are stored in `PyCapsule` instances, and exposed to the user as a standard Python class.

The `bind(c)` proceedures are meant to separate the implementation from the `C`-interface in a manner which can be automated (eventually) by `f2py`. Similarly, the `Python-C` code is eventually to be generated as well.

`point` with pointers

We will start with the prototypical class in applications, a `point`, with 2 dimensions.

Fortran baseline

The `type` we want is very minimal:

```
type :: Point
   real(8) :: x
   real(8) :: y
end type Point
```

However, since we need to use this via pointers, we need a more involved set of helpers for this:

```
subroutine create_point(x, y, p)
   real(8), intent(in) :: x, y
   type(Point), pointer :: p
   allocate (p)
   p%x = x
   p%y = y
end subroutine create_point
```

There isn't much to say about these, and they are rather mechanically derived.

Building and testing

For the toy point we have:

```
module point_module
   implicit none
   type :: Point
      real(8) :: x
      real(8) :: y
   end type Point
contains
   subroutine create_point(x, y, p, alloc_stat)
      real(8), intent(in) :: x, y
      type(Point), pointer :: p
      integer, intent(out), optional :: alloc_stat
      allocate (p, stat=alloc_stat)
      if (present(alloc_stat) .and. alloc_stat /= 0) then
         return
      end if
      p%x = x
      p%y = y
   end subroutine create_point

   subroutine destroy_point(p)
      type(Point), pointer :: p
      if (associated(p)) then
         deallocate (p)
         p => null()
      end if
   end subroutine destroy_point

   function get_x(p) result(x_value)
      type(Point), intent(in) :: p
      real(8) :: x_value
      x_value = p%x
   end function get_x

   subroutine set_x(p, x)
      type(Point), intent(inout) :: p
      real(8), intent(in) :: x
      p%x = x
   end subroutine set_x

   function get_y(p) result(y_value)
      type(Point), intent(in) :: p
      real(8) :: y_value
      y_value = p%y
   end function get_y

   subroutine set_y(p, y)
      type(Point), intent(inout) :: p
      real(8), intent(in) :: y
      p%y = y
   end subroutine set_y
end module point_module
```

Now these can be tested from a simple driver:

```
program test_point_module
   use point_module
   implicit none

   type(Point), pointer :: pt
   real(8) :: x_val, y_val
   integer :: alloc_status

   ! Test creating a new point
   call create_point(1.0d0, 2.0d0, pt, alloc_status)

   if (.not. associated(pt)) then
      print *, "Point creation failed with status ", alloc_status
      stop
   else
      print *, "Point created: (", pt%x, ", ", pt%y, ")"
   end if

   ! Test setters and getters
   call set_x(pt, 5.0d0)
   call set_y(pt, 10.0d0)
   x_val = get_x(pt)
   y_val = get_y(pt)

   print *, "After setting new values:"
   print *, "x =", x_val, "y =", y_val

   if (x_val == 5.0d0 .and. y_val == 10.0d0) then
      print *, "Point values updated correctly."
   else
      print *, "Error in updating point values."
   end if

   ! Test destruction
   call destroy_point(pt)

   if (.not. associated(pt)) then
      print *, "Point destroyed successfully."
   else
      print *, "Error in destroying point."
   end if
end program test_point_module
```

Personally I use `meson` for everything, but this is trivially compiled anyway:

```
❯ gfortran point.f90 ftest.f90
❯ ./a.out
 Point created: (   1.0000000000000000      ,    2.0000000000000000      )
 After setting new values:
 x =   5.0000000000000000      y =   10.000000000000000
 Point values updated correctly.
 Point destroyed successfully.
```

Just to build on for when it gets less trivial, the `meson` equivalent is:

```
project('pyclass_bindc', 'c', 'fortran',
        version : '0.1',
        default_options : ['warning_level=2',
                           'buildtype=debug',
                           'debug=true',
                          ])
executable('ftest',
           'ftest.f90',
           'point.f90')
```

Which can be used with:

```
meson setup bbdir
meson compile -C bbdir
❯ ./bbdir/ftest
 Point created: (   1.0000000000000000      ,    2.0000000000000000      )
 After setting new values:
 x =   5.0000000000000000      y =   10.000000000000000
 Point values updated correctly.
 Point destroyed successfully.
```

`C`-Wrappers

Lets focus a bit on what we need to do, creation and destruction are fairly straightforward since we just need to remember to return a `c_ptr` and create interface it to our existing functions..

```
function c_new_point(x, y) result(cnewpt) bind(c)
   real(c_double), value :: x, y !! important
   type(c_ptr) :: cnewpt
   type(Point), pointer :: p
   integer :: stat
   call create_point(x, y, p, stat)
   if (stat /= 0) then
      print *, "Allocation failed"
      cnewpt = C_NULL_PTR
      return
   end if
   cnewpt = c_loc(p)
end function c_new_point
```

Note the use of the `c_loc` intrinsic which is used to get a pointer compatible with `C` as required by the return type too. For bindings, it is never a good idea to neglect error handling, so we do guard for allocation failures, using the NULL helper in the `iso_c` intrinsic. The `value` attribute is important enough (and a recurring theme) so it is relegated to the box.

Warning
The `value` attribute is **required** in the to ensure data is copied and interpreted correctly from `C` to Fortran.
Similar considerations apply to the rest of the bindings, the full source of which is reproduced at the end of this section.

To call these from `C` we need to remember that we want to grab a pointer to a pointer, that is, the signatures will be `void**`. So we have:

```
extern void *c_new_point(double x, double y);
extern void c_delete_point(void **p);
extern double c_get_x(void **p);
extern void c_set_x(void **p, double x);
extern double c_get_y(void **p);
extern void c_set_y(void **p, double y);
```

`bind(C)` is on the Fortran side is used to ensure no name mangling.

Building and testing

The additional wrapper file is:

```
! cwrappers.f90
! C interoperable wrappers
module c_wrappers
   use, intrinsic :: iso_c_binding, only: c_ptr, c_double, c_null_ptr, &
                                                                        c_loc, c_associated, c_f_pointer
   use point_module
   implicit none

contains

   ! Wrapper for creating a new point
   function c_new_point(x, y) result(cnewpt) bind(c)
      real(c_double), value :: x, y
      type(c_ptr) :: cnewpt
      type(Point), pointer :: p
      integer :: stat
      call create_point(x, y, p, stat)
      if (stat /= 0) then
         print *, "Allocation failed"
         cnewpt = C_NULL_PTR
         return
      end if

      cnewpt = c_loc(p)
   end function c_new_point

   ! Wrapper to set the x value of a point
   subroutine c_set_x(p_cptr, x) bind(c)
      type(c_ptr), intent(in) :: p_cptr
      real(c_double), intent(in), value :: x
      type(Point), pointer :: p

      call c_f_pointer(p_cptr, p)
      call set_x(p, x)
   end subroutine c_set_x

   ! Wrapper to get the x value of a point
   function c_get_x(p_cptr) bind(c) result(x)
      type(c_ptr), intent(in) :: p_cptr
      real(c_double) :: x
      type(Point), pointer :: p

      call c_f_pointer(p_cptr, p)
      x = get_x(p)
   end function c_get_x

   ! Wrapper to set the y value of a point
   subroutine c_set_y(p_cptr, y) bind(c, name='c_set_y')
      type(c_ptr), intent(in) :: p_cptr
      real(c_double), intent(in), value :: y
      type(Point), pointer :: p

      call c_f_pointer(p_cptr, p)
      call set_y(p, y)
   end subroutine c_set_y

   ! Wrapper to get the y value of a point
   function c_get_y(p_cptr) bind(c, name='c_get_y') result(y)
      type(c_ptr), intent(in) :: p_cptr
      real(c_double) :: y
      type(Point), pointer :: p

      call c_f_pointer(p_cptr, p)
      y = get_y(p)
   end function c_get_y
```

```
   ! Wrapper for deallocating a point
   subroutine c_delete_point(p_cptr) bind(c, name='c_delete_point')
      type(c_ptr), intent(inout) :: p_cptr
      type(Point), pointer :: p

      if (.not. c_associated(p_cptr)) return
      call c_f_pointer(p_cptr, p)
      call destroy_point(p)
      p_cptr = c_null_ptr
   end subroutine c_delete_point

end module c_wrappers
```

With the corresponding `C` file to test:

```
#include <stdio.h>

/* Declarations of Fortran functions */
extern void *c_new_point(double x, double y);
extern void c_delete_point(void **p);
extern double c_get_x(void **p);
extern void c_set_x(void **p, double x);
extern double c_get_y(void **p);
extern void c_set_y(void **p, double y);

int main() {
  void *point = c_new_point(3.0, 4.0);
  if (point == NULL) {
    fprintf(stderr, "Failed to create point\n");
    return 1;
  }
  /* Retrieve the x and y values using the Fortran function */
  double x = c_get_x(&point);
  double y = c_get_y(&point);

  printf("Point coordinates: x = %f, y = %f\n", x, y);

  /* Set the x and y values using the Fortran function */
  c_set_x(&point, 5.0);
  c_set_y(&point, 6.0);

  printf("Point coordinates after set: x = %f, y = %f\n", c_get_x(&point),
         c_get_y(&point));

  /* Clean up and delete the point using the Fortran function */
  c_delete_point(&point);

  return 0;
}
```

Which can be compiled and run with the following addition to the existing `meson.build`:

```
executable('ctestf',
           'testc.c',
           'point.f90',
           'cwrappers.f90')
```

With:

```
meson compile -C bbdir
./bbdir/ctestf
Point coordinates: x = 3.000000, y = 4.000000
Point coordinates after set: x = 5.000000, y = 6.000000
```

Python-C Interface

The key take-away here is to use a PyCapsule object which:

> *represents an opaque value, useful for C extension modules who need to pass an opaque value (as a void* pointer) through Python code to other C code.*

In our case the opaque value needs to travel a bit further along and go through to Fortran and back, but the principle (and implementation) is the same. The `C` function calls will be the same, the only difference being the structure used, unlike in "shadowing" approaches, the `struct` will have neither `C` interop nor `Fortran` information. So we start with:

```
/* Point object definition */
typedef struct {
  PyObject_HEAD
  PyObject *capsule;
} PyPointObject;
```

This is simplicity itself. Naturally we need to actually grab the right object from the capsule before use. Additionally, there are a few nuances, e.g. before defining creation we need to define cleanup:

```
void point_capsule_destructor(PyObject *capsule) {
  void *point = PyCapsule_GetPointer(capsule, "point._Point");
  if (point) {
    c_delete_point(&point);
  }
}
```

Creation is simply via `PyCapsule_New` once appropriate steps (unpacking, error handling) are taken.

```
static PyObject *PyPoint_new(PyTypeObject *type, PyObject *args,
                             PyObject *kwds) {
  double x, y;
  if (!PyArg_ParseTuple(args, "dd", &x, &y)) {
    return NULL;
  }

  PyPointObject *self = (PyPointObject *)type->tp_alloc(type, 0);
  if (self != NULL) {
    void *point = c_new_point(x, y);
    if (point == NULL) {
      Py_DECREF(self);
      return PyErr_NoMemory();
    }
    self->capsule =
        PyCapsule_New(point, "point._Point", point_capsule_destructor);
    if (self->capsule == NULL) {
      c_delete_point(&point);
      Py_DECREF(self);
      return NULL;
    }
  }
  return (PyObject *)self;
}
```

Since the rest of this is standard, including `PyFloat_FromDouble` and the rest of the C-Python API, we will move onto the building and testing phase.

Building and testing

The full file, including boilerplate defining the module (detailed in earlier posts) is:

```
#ifndef PY_SSIZE_T_CLEAN
#define PY_SSIZE_T_CLEAN
#include <stdio.h>
#endif /* PY_SSIZE_T_CLEAN */

#include "Python.h"
#include "point.h"
#include "structmember.h"

void point_capsule_destructor(PyObject *capsule) {
  void *point = PyCapsule_GetPointer(capsule, "point._Point");
  if (point) {
    c_delete_point(&point);
  }
}

/* Point object definition */
typedef struct {
  PyObject_HEAD
  PyObject *capsule;
} PyPointObject;

/* Deallocates the PyPointObject */
static void PyPoint_dealloc(PyPointObject *self) {
  Py_XDECREF(self->capsule);
  Py_TYPE(self)->tp_free((PyObject *)self);
}

/* Creates a new PyPointObject */
static PyObject *PyPoint_new(PyTypeObject *type, PyObject *args,
                             PyObject *kwds) {
  double x, y;
  if (!PyArg_ParseTuple(args, "dd", &x, &y)) {
    return NULL;
  }

  PyPointObject *self = (PyPointObject *)type->tp_alloc(type, 0);
  if (self != NULL) {
    void *point = c_new_point(x, y);
    if (point == NULL) {
      Py_DECREF(self);
      return PyErr_NoMemory();
    }
    self->capsule =
        PyCapsule_New(point, "point._Point", point_capsule_destructor);
    if (self->capsule == NULL) {
      c_delete_point(&point);
      Py_DECREF(self);
      return NULL;
    }
  }
  return (PyObject *)self;
}

/* Getter for the 'x' attribute */
static PyObject *PyPoint_getx(PyPointObject *self, void *closure) {
  void *point = PyCapsule_GetPointer(self->capsule, "point._Point");
  double x = c_get_x(&point);
  return PyFloat_FromDouble(x);
}

/* Setter for the 'x' attribute */
static int PyPoint_setx(PyPointObject *self, PyObject *value, void *closure) {
  if (value == NULL) {
    PyErr_SetString(PyExc_TypeError, "Cannot delete the x attribute");
```

```
    return -1;
  }
  if (!PyFloat_Check(value)) {
    PyErr_SetString(PyExc_TypeError, "The x attribute value must be a float");
    return -1;
  }
  double x = PyFloat_AsDouble(value);
  void *point = PyCapsule_GetPointer(self->capsule, "point._Point");
  c_set_x(&point, x);
  return 0;
}

/* Getter for the 'y' attribute */
static PyObject *PyPoint_gety(PyPointObject *self, void *closure) {
  void *point = PyCapsule_GetPointer(self->capsule, "point._Point");
  double y = c_get_y(&point);
  return PyFloat_FromDouble(y);
}

/* Setter for the 'y' attribute */
static int PyPoint_sety(PyPointObject *self, PyObject *value, void *closure) {
  if (value == NULL) {
    PyErr_SetString(PyExc_TypeError, "Cannot delete the y attribute");
    return -1;
  }
  if (!PyFloat_Check(value)) {
    PyErr_SetString(PyExc_TypeError, "The y attribute value must be a float");
    return -1;
  }
  double y = PyFloat_AsDouble(value);
  void *point = PyCapsule_GetPointer(self->capsule, "point._Point");
  c_set_y(&point, y);
  return 0;
}

/* Define the properties of the Point type */
static PyGetSetDef PyPoint_getseters[] = {
    {"x", (getter)PyPoint_getx, (setter)PyPoint_setx, "x coordinate", NULL},
    {"y", (getter)PyPoint_gety, (setter)PyPoint_sety, "y coordinate", NULL},
    {NULL} /* Sentinel */
};

/* Initializes the PyPointType object */
static PyTypeObject PyPointType = {
    PyVarObject_HEAD_INIT(NULL, 0).tp_name = "point.Point",
    .tp_doc = "Point objects",
    .tp_basicsize = sizeof(PyPointObject),
    .tp_itemsize = 0,
    .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE,
    .tp_new = PyPoint_new,
    .tp_dealloc = (destructor)PyPoint_dealloc,
    .tp_getset = PyPoint_getseters,
};

static PyMethodDef point_methods[] = {
    {NULL, NULL, 0, NULL} /* Sentinel */
};

/* Initializes the point module */
static PyModuleDef pointmodule = {
    PyModuleDef_HEAD_INIT,
    .m_name = "point",
    .m_doc = "Module that provides a Point object.",
    .m_size = -1,
    .m_methods = point_methods,
};
```

```
PyMODINIT_FUNC PyInit_point(void) {
  if (PyType_Ready(&PyPointType) < 0) {
    return NULL;
  }

  PyObject *m = PyModule_Create(&pointmodule);
  if (m == NULL) {
    return NULL;
  }

  Py_INCREF(&PyPointType);
  if (PyModule_AddObject(m, "Point", (PyObject *)&PyPointType) < 0) {
    Py_DECREF(&PyPointType);
    Py_DECREF(m);
    return NULL;
  }

  return m;
}
```

Which needs some additions to the `meson.build` [^fn:3]

```
py_mod = import('python')
py3 = py_mod.find_installation('python3')
py3_dep = py3.dependency()
message(py3.path())
message(py3.get_install_dir())

# Python Class Representation
py3.extension_module('point',
           'point.f90',
           'cwrappers.f90',
           'pointcapsule.c',
           dependencies : py3_dep,
           install : true)
```

With a simple `pytest` in `tests/test_simple.py`:

```
from point import Point

def test_point():
    p = Point(1, 2)
    assert p.x == 1
    assert p.y == 2
    assert p != Point(1, 2)
```

Can be compiled and run with:

```
meson compile -C bbdir
export PYTHONPATH=$PYTHONPATH:$(pwd)/bbdir
pytest
```

Note that we have not (yet) defined any type bound proceedures, so there are no operations defined between `Point` objects or anything else.

Binding type bound proceedures

Given the structure discussed so far, adding a type bound proceedure is nearly smooth. We need to modify the type definition.

```
type :: Point
   real(8) :: x
   real(8) :: y
contains
   procedure, pass(self) :: euclidean_distance
end type Point
```

Followed by adding a “handle” for it:

```
real(8) function euclidean_distance(self, other) result(distance)
   class(Point), intent(in) :: self, other
   distance = sqrt((self%x - other%x)**2 + (self%y - other%y)**2)
end function euclidean_distance
```

Along with a wrapper:

```
function c_euclidean_distance(this_cptr, other_cptr) bind(c) result(y)
   type(c_ptr), value :: this_cptr, other_cptr
   real(c_double) :: y
   type(Point), pointer :: p1, p2
   call c_f_pointer(this_cptr, p1)
   call c_f_pointer(other_cptr, p2)
   y = p1%euclidean_distance(p2)
end function c_euclidean_distance
```

Which can be called from `C` via:

```
extern double c_euclidean_distance(void *p1, void *p2);
```

Finally the `python-C` code requires only the direct addition of the method:

```
static PyObject *PyPoint_euclidean_distance(PyPointObject *self,
                                            PyObject *args) {
  PyObject *other;
  if (!PyArg_ParseTuple(args, "O", &other)) {
    return NULL;
  }

  /* Check if 'other' is a PyPointObject instance by comparing type names */
  if (strcmp(other->ob_type->tp_name, "point.Point") != 0) {
    PyErr_SetString(PyExc_TypeError, "Argument must be a Point instance");
    return NULL;
  }

  /* Ensure 'other' is a capsule before getting the pointer */
  if (!PyCapsule_CheckExact(((PyPointObject *)other)->capsule)) {
    PyErr_SetString(PyExc_TypeError, "Argument is not a Point capsule");
    return NULL;
  }

  void *self_point = PyCapsule_GetPointer(self->capsule, "point._Point");
  void *other_point =
      PyCapsule_GetPointer(((PyPointObject *)other)->capsule, "point._Point");

  if (!self_point || !other_point) {
    PyErr_SetString(PyExc_RuntimeError, "Failed to get Point from capsule");
    return NULL;
  }

  double distance = c_euclidean_distance(self_point, other_point);
  return PyFloat_FromDouble(distance);
}

/* Define the methods */
static PyMethodDef point_methods[] = {
    {"euclidean_distance", (PyCFunction)PyPoint_euclidean_distance,
     METH_VARARGS, "Calculate the Euclidean distance to another Point"},
    {NULL, NULL, 0, NULL} /* Sentinel */
};
```

This can now be used to pass a more stringent set of tests:

```
from point import Point

def test_point():
    p = Point(1, 2)
    assert p.x == 1
    assert p.y == 2
    assert p != Point(1, 2)

def test_point_euclid():
    p = Point(1, 2)
    p2 = Point(2, 5)
    assert p.euclidean_distance(p2) == 3.1622776601683795

def test_fails():
    p = Point(1, 2)
    with pytest.raises(TypeError) as e_info:
        p.euclidean_distance(1)
    assert str(e_info.value) == "Argument must be a Point instance"
```

Towards allocatable components and `f2py` support

So far we have not really covered new ground compared to the previous iteration using `C` compatible `struct` information. The remarkable extensibility of the pointer approach really shines when we consider adding a new type, with an allocatable component[^fn:4].

That being said, there is more to be done over on the `f2py` side before it makes sense to dig too deeply into enhancements on the `Fortran-C-Python` pipeline. The biggest blocker at the moment is:

- Suport for `C_PTR` in `f2py`
  - This includes `void*` handling

The exact details shall be covered later, mostly because the Python-C code gets a little rough. It is easier with `f2py` which provides efficient routines to and from `C` and Python via `numpy` arrays but still too long to be in the same post.

Conclusions

More examples notwithstanding, the design discussed here is generic enough to form a more coherent and pragmatic basis for further improvements. Some of these are enumerated below for a future attempt:

- Circumventing the need to copy data from Python / `C` to Fortran [^fn:5]
  - Either by careful usage of `transfer` or just via `c_loc` with compatible types
- Using the `CFI` descriptors to operate efficiently on discontinous memory [^fn:6]
  - Actually generally using the `CFI` descriptors more, since 2018 Reid (2018), some of the pointer highjinks are enshrined as cannon
    - `c_loc` and `c_funloc` may have non-interoperable arguments Metcalf, Reid, and Cohen (2018)

All that being said, with `meson` support in `f2py` from `1.26` (npgh-24532), basic types based modern Fortran can already be used, so the future remains bright.

Reading list

Fortran derived types, interoperability, and the references for wrapping them, collected as a citation list: 3 references. The collection exports to BibTeX or any CSL style, so it can go straight into a bibliography.

References

Metcalf, Michael, John Ker Reid, and Malcolm Cohen. 2018. *Modern Fortran Explained: Incorporating Fortran 2018*. Numerical Mathematics and Scientific Computation. Oxford [England]: Oxford University Press.
Pletzer, Alexander, Douglas McCune, Stefan Muszala, Srinath Vadlamani, and Scott Kruger. 2008. "Exposing Fortran Derived Types to C and Other Languages." *Computing in Science Engineering* 10 (4): 86--92. <https://doi.org/10.1109/MCSE.2008.94>.
Reid, John. 2018. "The New Features of Fortran 2018." <https://doi.org/10.1145/3206214.3206215>.

```
alternatives](https://fortran-lang.discourse.group/t/best-way-to-declare-a-double-precision-in-fortran/69/21?
u=rgoswami)
```

```
`meson` pays off over hand compilation
```

```
thread](https://fortran-lang.discourse.group/t/allocate-interoperability-and-c-descriptors/5088/9?u=rgoswami)
includes examples from Ivan Pribec demonstrating this
```


```
thread](https://fortran-lang.discourse.group/t/allocate-interoperability-and-c-descriptors/5088/9?u=rgoswami)
includes examples from Ivan Pribec demonstrating this
```


```
interactive workflows
```